LeetCode Practice

2 Easy: Sliding Window

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

2
EASY

Patterns (in order)

1. Sliding Window (2)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

2 questions | tap a title to jump to it

Easy questions (2) Go >

Sliding Window (2) Go >

E 1. Contains Duplicate II LC 219

E 2. Maximum Average Subarray I LC 643

Easy questions (2)

EASY | TOPIC 1 OF 1

Sliding Window

2 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Contains Duplicate II LC 219
2. Maximum Average Subarray I LC 643

EASY | Sliding Window | LeetCode #219 | 1 of 2 | 1/2

Contains Duplicate II

QUESTION

Given an integer array nums and an integer k, return true if there are two distinct indices i and j in the array such that nums[i] == nums[j] and abs(i - j) <= k.

Example 1	
Input	`nums = [1,2,3,1], k = 3`
Output	`true`

Example 2	
Input	`nums = [1,0,1,1], k = 1`
Output	`true`

Example 3	
Input	`nums = [1,2,3,1,2,3], k = 2`
Output	`false`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 10^9 <= nums[i] <= 10^9
- 0 <= k <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return true if any two distinct indices i and j exist such that nums[i] == nums[j] AND abs(i-j) <= k. Right?"

"A few quick questions:

Distinct indices means i != j? Yes. Can k be 0? Yes — then no pair qualifies."

"Let me walk: nums=[1,2,3,1], k=3 -> indices 0 and 3 have same value, diff=3<=3 -> true ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_i_+_k_+_1):
        if nums[i] == nums[j]:
            return True
```

TIME	SPACE
O(n * k)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Checks k neighbours of every element. A hash map from value to its last index (or a set holding the last k values) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is checking all pairs within range k.

Sliding window hash set of size k:

Maintain a set of the last k elements. For each new element, if it's already in the set -> found.

Remove the element that falls out of the window.

Time: O(n). Space: O(k).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _219_ContainsDuplicateII:
    def containsNearbyDuplicate(self, nums, k):
        window = set()
        for i, num in enumerate(nums):
            if num in window:
                return True
            window.add(num)
            if len(window) > k:
                window.discard(nums[i - k])
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace the expand/contract window and show how it preserves the condition."

nums=[1,2,3,1], k=3:

i=0: window={1}. i=1: window={1,2}. i=2: window={1,2,3}. i=3(1): 1 in window -> True ok

nums=[1,0,1,1], k=1:

i=0: window={1}. i=1(0): add, window={1,0}, size>1->remove nums[0]=1. window={0}.

i=2(1): not in window, add. window={0,1}. i=3(1): 1 in window -> True ok

Removing nums[i-k] when window exceeds k correctly maintains the size-k window

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) each element added/removed at most once	O(k) sliding window

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: Contains Duplicate I (#217) — k=n case; any duplicate anywhere.

Follow-up: Contains Duplicate III (#220) — abs value diff <= t; needs sorted structure.

IN YOUR APP | Expose | Editorial | Community

EASY | Sliding Window | LeetCode #643 | 2 of 2 | 2/2

Maximum Average Subarray I

QUESTION

You are given an integer array nums consisting of n elements, and an integer k.

Find a contiguous subarray whose length is equal to k that has the maximum average value and return this value. Any answer with a calculation error less than 10^{-5} will be accepted.

Example 1	
Input	`nums = [1,12,-5,-6,50,3], k = 4`
Output	`12.75000`
Explanation	*Maximum average is (12 - 5 - 6 + 50) / 4 = 51 / 4 = 12.75*

Example 2	
Input	`nums = [5], k = 1`
Output	`5.00000`

CONSTRAINTS

- n == nums.length
- 1 <= k <= n <= 10^5
- 10^4 <= nums[i] <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the maximum average of any contiguous subarray of length exactly k. Return the average. Right?"

"A few quick questions:

k <= n always? Yes. Values can be negative? Yes."

"Let me walk: nums=[1,12,-5,-6,50,3], k=4 -> windows sums: 2,51,42. max=51. avg=51/4=12.75 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in is:
    sum the k elements
    best = max(best, average)
```

TIME	SPACE
O(n * k)	O(1)

WHY IT IS SLOW

Re-sums k elements for every window. Slide the window: add the entering element, subtract the leaving one: O(n).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is recomputing the window sum from scratch each step.

Sliding window: compute the first window sum, then slide by adding the new element

and subtracting the element that falls out.

Time: O(n). Space: O(1)."

Key insight: adding the incoming element and subtracting the outgoing element updates the window sum in O(1) — one pass replaces O(k) partial sums per step.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _643_MaxAverageSubarrayI:
    def findMaxAverage(self, nums, k):
        window_sum = sum(nums[:k])
        max_sum = window_sum
        for i in range(k, len(nums)):
            window_sum += nums[i] - nums[i - k]
            max_sum = max(max_sum, window_sum)
        return max_sum / k
```

PHASE 5 TEST & DEBUG

"Let me trace the expand/contract window and show how it preserves the condition."

nums=[1,12,-5,-6,50,3], k=4:

window_sum=1+12-5-6=2, max=2.

i=4: +=50-1=49, sum=51, max=51. i=5: +=3-12=-9, sum=42, max=51. -> 51/4=12.75 ok

nums=[5], k=1: sum=5, no slide. -> 5.0 ok

Sliding update adds new element and removes element that exited the window

WHY: DP/greedy isn't needed because the window property is maintained by expanding and contracting while keeping the condition valid.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) one initial sum + n-k slide steps	O(1)

Follow-up: Maximum Average Subarray II (#644) — any length >= k; binary search on answer.

Follow-up: minimum average subarray — same sliding window, track min instead of max.

IN YOUR APP | Expose | Editorial | Community